

Ceedling and Jenkins

Lab 9

Add ceedling unit tests to the atcmd project and configure your pipeline to run them automatically. Starting point for the project is given in the attached zip-file. The structure of the project has changed a bit from the previous version to keep the sources and pipeline/test files in separate directories. The project has a new mode of operation that can be entered with command 'ATO'. On receiving command 'ATO' the program immediately switches to online mode where all received characters are transformed the same way as text parameter of AT+SEND command and sent using morse code. In this mode the normal AT-commands have no effect. To exit from the online mode a special escape sequence is needed. The sequence is:

1. No characters are transmitted for at least one second.
2. Three plus ('+') characters are sent with less than 500ms intervals. No other characters are sent (= no linefeed)
3. No characters are transmitted for at least one second.

When device exits online mode it responds with 'OK'.

Add Robot framework test that verifies that you can enter and exit online mode.

All your previous Robot framework tests must also work! Modify the previous tests to use dot lengths between 10 and 30 ms to make the tests run faster.

Add unit tests for two files:

- readchar.c
- echo.c

Readchar.c contains a function for reading characters from standard input with a timeout and special handling for an escape sequence. For this file you must implement at least one test for the correct escape sequence (a few timeouts, followed by 3 x '+', and a few timeouts) and one test for incorrect sequence.

Echo.c does not have a working implementation of the functions introduced in the headers. These functions should be implemented using TDD, so you start by writing a failing test and then add just enough code to make the test pass, etc. Echo.c should use functions write_wds and read_wds to store/retrieve the state of local echo. The first parameter is storage location address in range 0-3. Each location can store one 32-bit value. The values are retained across reset by debugger or reset button. Powering off device will lose the stored data. The functions defined in hw.c are to be mocked by ceedling so that calls to those functions are replaced with the test system functions.

Show the instructor that your tests work.

Instructions

General information about ceedling can be found in the workspace:

Documents/How+to+Use+Ceedling+for+Embedded+Test-Driven+Development.pdf

Start by adding the unit tests and run them manually on your computer. When the tests work then you can change the pipeline to run unit tests automatically.

Do the following preparations (see instructions after the list for more details):

- Unzip atcmd_ceedling.zip
 - The zip has recommended directory structure for the sources
- Create a ceedling project in atcmd directory.
 - You should be in the atcmd's parent directory when you create the ceedling project. That ensures that directories are created under atcmd
- Test that you can execute the (non-existent) tests.
- Test that you can build the updated project on your computer.

Adding unit tests

The easiest way to run ceedling is to run it in a container and mount the local folder containing test to the container to give container access to the sources under test.

First, we need to create a ceedling project in the morse sender folder.

How to run ceedling in a container: <https://github.com/ThrowTheSwitch/MadScienceLabDocker>

The command given in the link above will bind mount your project to the container. WSL2 docker on windows may warn about performance. Mount the parent directory of your project to the container.

Create a ceedling project in atcmd folder. It already contains src folder and creating a ceedling project will add the rest of the folders needed by ceedling.

This is the current directory. Atcmd should be subdirectory to this directory.

Command to run project creation:

```
docker run --rm -v %cd%:/project throwtheswitch/madsciencelab:0.3.2 /bin/sh -c  
"ceedling new atcmd"
```

Windows

```
docker run --rm -v ${PWD}:/project throwtheswitch/madsciencelab:0.3.2 /bin/sh -c  
"ceedling new atcmd"
```

Linux/OSX

```
Welcome to Ceedling!
```

```
create atcmd/project.yml
```

```
Project 'atcmd' created!
```

```
- Execute 'ceedling help' from atcmd to view available test & build tasks
```

Project creation adds files and folders in Atcmd.

You can use ceedling.bat you extracted from the zip-file to run your ceedling tests.

```
>ceedling.bat  
  
>docker run --rm -v C:\Users\lansk\Documents\tmp\ceedlingdemo\atcmd:/project throwtheswitch/madsciencelab:0.3.2 /bin/sh -c "ceedling test:all"
```

```
-----  
OVERALL TEST SUMMARY  
-----
```

```
No tests executed.
```

```
>
```

When you run ceedling.bat immediately after creating the ceedling project the result should say “No tests executed”.

This is a good time to create a new repository in gogs and add your sources and tests to the repository. Note that build and cmake-build-debug directories should not be added to the repository. They are automatically created by ceedling or pico-build. See:

<https://github.com/github/gitignore/blob/main/Global/JetBrains.gitignore> and

<https://github.com/github/gitignore/blob/main/CMake.gitignore> for .gitignore examples.

When you have a new repository set up then you can start working on the unit tests.

Tests are written in C files. Pictures are from a past project. The directory structure and the test concepts remain the same in the current project.

The screenshot shows a GitHub repository interface. At the top, there are tabs for 'Files', 'Issues' (0), 'Pull Requests' (0), and 'Wiki', along with a 'Settings' link. Below the tabs, the repository path is 'morse / MorseSender / test'. The main content area displays a list of commits. The first commit is by 'Keijo Lämsikunnas' with commit hash 'a727eefdff', titled 'Fixed the test', and dated '11 months ago'. Below this, there is a list of files and their corresponding commits: 'support' (27296f1faf, 'Added ceedling tests', 11 months ago), 'test_morse.c' (27296f1faf, 'Added ceedling tests', 11 months ago), and 'test_morse_send.c' (a727eefdff, 'Fixed the test', 11 months ago).

File	Commit Hash	Commit Message	Time
support	27296f1faf	Added ceedling tests	11 months ago
test_morse.c	27296f1faf	Added ceedling tests	11 months ago
test_morse_send.c	a727eefdff	Fixed the test	11 months ago

The tests use concepts that sound familiar to a Robot Framework user.

FilesIssues0Pull Requests0Wiki

Branch: master morse / MorseSender / test / test_morse_send.c

test_morse_send.c 829 B

```
1 #include "unity.h"
2
3 #include "morse_send.h"
4
5 #include "mock_morse.h"
6
7 void setUp(void)
8 {
9 }
10
11 void tearDown(void)
12 {
13 }
14
15 void test_morse_send_SOS(void)
16 {
17     morse_symbol_Expect('S', 300);
18     morse_symbol_Expect('O', 300);
19     morse_symbol_Expect('S', 300);
20     morse_send("SOS", 300);
21 }
22
23
```

FilesIssues0Pull Requests0Wiki

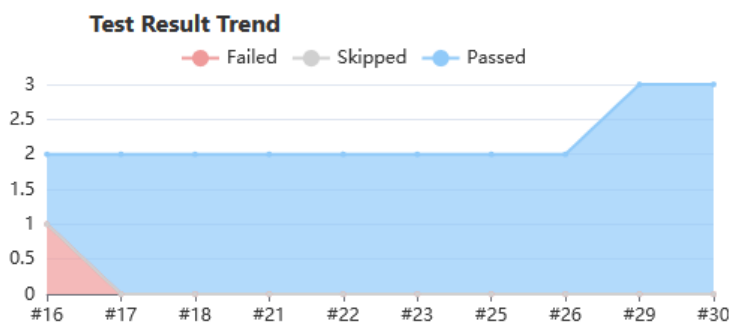
Branch: master morse / MorseSender / test / test_morse.c

test_morse.c 630 B

```
1 #include "unity.h"
2
3 #include "morse.h"
4
5 #include "FreeRTOSConfig.h"
6 #include "FreeRTOS.h"
7
8 #include "morse_codes.h"
9
10 #include "mock_wrappers.h"
11
12
13 void setUp(void)
14 {
15 }
16
17 void tearDown(void)
18 {
19 }
20
21 void test_morse_SendLetterA(void)
22 {
23     //vTaskDelay_Expect(100);
24     pin_write_Expect(1);
25     sleep_Expect(25);
26     pin_write_Expect(0);
27     sleep_Expect(25);
28     pin_write_Expect(1);
29     sleep_Expect(75);
30     pin_write_Expect(0);
31     sleep_Expect(25);
32     sleep_Expect(50);
33     morse_symbol_Expect('A', 25); // A = DOT, DASH
34     //TEST_IGNORE_MESSAGE("Need to Implement morse");
35 }
```

Add Ceedling to pipeline

Add Ceedling tests as a parallel stage to your pipeline so that pico-build and ceedling tests take place at the same time. In the stage make two steps: run ceedling and parse results. The parsed results will be displayed on the classic Jenkins view.

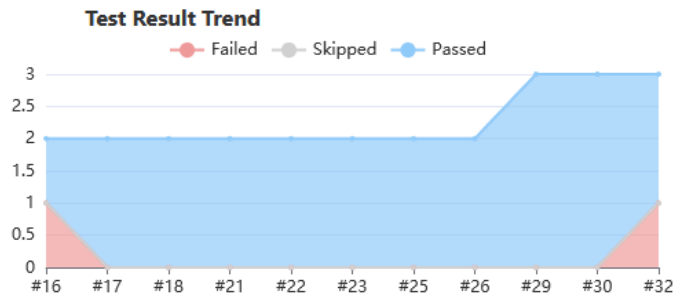


To parse the results, you need to **install xUnit plugin** to Jenkins, edit project.yml to enable xml-reports and download a parsing template (unity.xml) for ceedling's xml-results. Unity.xml is included in the attached zip.file.

See following link for details:

<https://embeddedartistry.com/blog/2019/02/25/unit-testing-and-reporting-on-a-build-server-using-ceedling-and-unity/> as usual with evolving software the blog is already a bit outdated. The path to the report is 'build/artifacts/**test**/report.xml '

Remember to test that report is produced even if a unit test fails and that skipped tests are also reported.



You don't need to add code coverage report to the Pipeline.